Delta encoding, version 2: the theory behind the compressor

A companion to the Sprint 4 deck, with references to the code

Dmitry Muravyev

About this document

The Sprint 4 deck shows the ideas of version 2 on slides. This document gives the same theory in full: every definition, every proof and the reasons behind the implementation choices. Starting from Lemma 1 it follows the slides one by one and goes into more detail than a slide can.

Each section ends with **In the code**, which names the methods that implement it. All of them are in `src/DeltaComp.Core/Compressor.cs` unless another file is named. The methods are referred to by name rather than by line number, because line numbers change with every edit; a search for the name finds them.

1. Notation

All sizes are in bits. The symbols below are used throughout; symbols needed in one section only are defined there.

Symbol	Meaning
N	number of frames (pixels or samples)
i	frame number, $0 \le i < N$
C	number of channels
c	channel number, $0 \le c < C$
$x_{c,i}$	value of channel c in frame i (at the target bit depth)
k_c	bits of channel c (target bit depth)
F	bits per frame, $F = \sum_c k_c$
b	bits of the block length field
q	side of the squares the image is split into ($q = 1$: no squares)
δ	Δ option: which windows the methods use
p	first frame of a block
l	block length in frames
$l_{max}(p)$	the longest possible block at p: $\min(2^b,\ N - p)$
m, w	method number; window width in bits
M_c, d_c	number of methods of channel c; bits of a method number
H	header of a block, $H = b + \sum_c d_c$
$L_w^F(p)$, $L_w^A(p)$	the longest block from p that the Fixed / Adaptive window of width w can encode
$L_{c,m}(p)$	the same for method m of channel c (one of the two above, or $l_{max}(p)$ for no compression)
$\mathrm{cost}_c(m, l)$	bits of channel c in a block of l frames encoded with method m
$\mathrm{cost}_c^*(p, l)$	the cheapest of them among the methods that can encode $p \dots p + l - 1$
$\mathrm{Bits}(p, l)$	size of the cheapest block of frames $p \dots p + l - 1$

Symbol	Meaning
$\mathrm{Best}(p)$	the smallest size of frames $p \dots N-1$

A block is written $p \dots p+l-1$: it starts at frame p and has l frames. When only one channel is discussed, the index c is dropped: x_i, k.

2. The data and the stream

2.1 Frames and squares

A frame is the set of all channel values of one pixel (or sample): $x_i = (x_{0,i}, \dots, x_{C-1,i})$. The values are taken at the target bit depth, after the channel transformations of the application (grayscale conversion, alpha, resizing, bit depth reduction).

An image can be split into $q \times q$ squares, where q divides both the width W and the height of the image. The squares go left to right, then top to bottom, and the pixels inside a square go row by row. Frame i is then taken from pixel $\pi(i)$:

$$t = \lfloor i/q^2 \rfloor, \quad o = i \bmod q^2, \quad r = \lfloor t/(W/q) \rfloor, \quad u = t \bmod (W/q)$$

$$\pi(i) = (rq + \lfloor o/q \rfloor)\, W + uq + (o \bmod q)$$

Here t is the number of the square, o the offset of the frame inside it, and r, u the row and the column of the square. Squares keep pixels that are close in the image close in the frame sequence, so neighboring frames differ less.

In the code: `GetPixelForSquares()` computes $\pi(i)$. `PrepareFrames()` reads every channel once into the array `_frames[c][i]` in this order, so the analysis never calls the channels again.

2.2 Block format

The output is a sequence of blocks. A block has a header and data:

1. the block length minus one, in b bits (so the length l is $1 \dots 2^b$);
2. the method number of every channel, d_c bits each;
3. the initial value of every compressed channel, k_c bits each (an uncompressed channel has none);
4. the data of the frames.

Every block is self-contained: the decoder needs nothing from the previous block. This is what makes the cost of a block independent of the other blocks, which the whole theory relies on.

The number of methods depends on the Δ option: $M_c = k_c + 1$ with one kind of window, $M_c = 2k_c$ with both kinds. A method number takes $d_c = \lfloor \log_2(M_c - 1) \rfloor + 1$ bits (at least 1). The header (the part that doesn't depend on the methods) is

$$H = b + \sum_{c=0}^{C-1} d_c.$$

In the code: `CalculateHeaderBits()` computes H and d_c (`BitsPerMethodDeclaration`); `GetMethodsCount()` gives M_c; `CompressBlock()` writes a block.

2.3 Methods and their costs

A method of a channel is one of: a Fixed window of width w ($0 \le w \le k-1$), an Adaptive floating window of width w ($1 \le w \le k-1$; the width 0 would be the same as the Fixed one), or no compression. The Δ option decides which of them exist and how they are numbered:

m	Fixed only	Adaptive only	Fixed first	Adaptive first
0	Fixed $w = 0$	Fixed $w = 0$	Fixed $w = 0$	Fixed $w = 0$

m	**Fixed only**	**Adaptive only**	**Fixed first**	**Adaptive first**
$1 \dots k-1$	Fixed $w = m$	Adaptive $w = m$	Fixed $w = m$	Adaptive $w = m$
k	none	none	none	none
$k+1 \dots 2k-1$	—	—	Adaptive $w = m - k$	Fixed $w = m - k$

The cost of one channel in a block of l frames (without the header H):

- **Fixed window** of width w: $\text{cost} = k + l\,w$. The initial value v is the minimum of the block, and every frame is written as $x_i - v$ in w bits. With $w = 0$ all frames equal v and the cost is just k.
- **Adaptive floating window** of width w: $\text{cost} = k + (l-1)\,w$. The initial value is the first frame, which needs no data of its own; every next frame is written relative to a window around the previous frame (section 3.2).
- **No compression**: $\text{cost} = l\,k$, no initial value.

All three are straight lines in l with a non-negative slope. This simple fact is used again and again below.

In the code: `GetRealMethod()` turns a method number into the window and its kind, `GetCompressedBitsForMethod()` is the cost formula.

2.4 Size of a block and of the stream

A block $p \dots p+l-1$ with the methods $\mathbf{m} = (m_0, \dots, m_{C-1})$ takes

$$\text{Bits}(p, l, \mathbf{m}) = H + \sum_{c=0}^{C-1} \text{cost}_c(m_c, l),$$

and method m_c may be used only if it can encode all l frames, which is $l \le L_{c,m_c}(p)$ (section 3). The stream is a chain of blocks covering frames $0 \dots N-1$, and the file has $\lceil \frac{1}{8} \sum_j \text{Bits}_j \rceil$ bytes. Rounding up to bytes is monotonic, so the smallest number of bits also gives the smallest file.

The compression efficiency of a block, $E = 1 - \text{Bits}/(lF)$, was the criterion of versions 1 (sprints 2–3). Version 2 doesn't use it for the chain at all: it minimizes the total size directly (section 5).

3. Windows and their lengths

3.1 Fixed window

The Fixed window of width w can encode the frames $p \dots p+l-1$ if all of them fit between the minimum v of the block and $v + 2^w - 1$:

$$\max_{p \le i < p+l} x_i - \min_{p \le i < p+l} x_i \le 2^w - 1.$$

The longest such block is

$$L^F_w(p) = \max\{\, l \le l_{max}(p) : \text{the range of } x_p \dots x_{p+l-1} \text{ is at most } 2^w - 1 \,\}.$$

3.2 Adaptive floating window

The Adaptive window of width 2^w is placed around the previous frame y and moved back inside the value range $0 \dots 2^k - 1$ if it sticks out. Its lowest value is

$$\text{lo}_w(y) = \min\big(\max(y - 2^{w-1},\ 0),\ 2^k - 2^w\big),$$

and frame x_i fits if $\mathrm{lo}_w(x_{i-1}) \le x_i < \mathrm{lo}_w(x_{i-1}) + 2^w$. It is written as $x_i - \mathrm{lo}_w(x_{i-1})$ in w bits. The decoder knows x_{i-1} (it has just decoded it) and calculates the same window, so the window is never stored. The first frame of the block is the initial value. So

$$L_w^A(p) = \max\{\, l \le l_{max}(p) : x_{p+1}, \dots, x_{p+l-1} \text{ each fit the window around the frame before it} \,\}.$$

Because the compression is lossless, the "previous frame" is the original value x_{i-1}. It doesn't depend on w or on anything chosen by the compressor. This is what Lemma 1 needs.

3.3 Lemma 1: nested windows

Lemma 1. For every start frame p and every channel,

$$L_0^F(p) \le L_1^F(p) \le \ldots \le L_{k-1}^F(p), \qquad L_1^A(p) \le L_2^A(p) \le \ldots \le L_{k-1}^A(p).$$

Proof for the Fixed window. The condition "range $\le 2^w - 1$" only gets weaker as w grows, so every block that the width w can encode, the width $w+1$ can encode too.

Proof for the Adaptive window. It is enough to show that for the same previous frame y the window of width 2^{w+1} contains the window of width 2^w, i.e.

$$\mathrm{lo}_{w+1}(y) \le \mathrm{lo}_w(y) \quad \text{and} \quad \mathrm{lo}_w(y) + 2^w \le \mathrm{lo}_{w+1}(y) + 2^{w+1}.$$

Then every frame that fits the narrow window fits the wide one, and since the previous frames are the same source values for both widths, the wide window lasts at least as long. Let $h = 2^{w-1}$ and $R = 2^k$, so that $\mathrm{lo}_w(y) = \mathrm{clamp}(y - h,\ 0,\ R - 2h)$ and $\mathrm{lo}_{w+1}(y) = \mathrm{clamp}(y - 2h,\ 0,\ R - 4h)$, where $\mathrm{clamp}(t, 0, U) = \min(\max(t, 0), U)$. (Both upper bounds are non-negative: the widest Adaptive window is 2^{k-1}, so $4h \le R$.)

- *The first inequality.* clamp grows with t and with U, and both are smaller for the wider window ($y - 2h \le y - h$, $R - 4h \le R - 2h$).
- *The second inequality,* $\mathrm{lo}_w(y) - \mathrm{lo}_{w+1}(y) \le 2h$, case by case for the wide window:
  - it is clamped at 0, so $y - 2h \le 0$: then $\mathrm{lo}_w(y) \le \max(y - h, 0) \le h \le 2h$;
  - it is clamped at the top, $\mathrm{lo}_{w+1}(y) = R - 4h$: then $\mathrm{lo}_w(y) \le R - 2h = \mathrm{lo}_{w+1}(y) + 2h$;
  - it is not clamped, $\mathrm{lo}_{w+1}(y) = y - 2h$: then $\mathrm{lo}_w(y) \le y - h = \mathrm{lo}_{w+1}(y) + h$. ∎

(The same was also checked by brute force for all y, all w and all $k \le 12$.)

Consequence: one scan for all widths. Walk from frame p forward and keep the narrowest window that still fits all frames so far. When a frame doesn't fit, the current window (and maybe a few wider ones) ends there: their lengths are known, and the scan continues with the next width that fits. By Lemma 1 no window ever has to be restarted, so one scan of at most $l_{max}(p)$ frames gives the lengths of all k windows. Version 1 scanned the data once per window: $O(k \cdot 2^b)$ per channel and position instead of $O(2^b + k)$.

In the code: `GetMaxBlockLengths()` (Fixed) and `GetMaxBlockLengths3()` (Adaptive) are the one-scan versions. `GetMaxBlockLength()` and `GetMaxBlockLength3()` are the old one-window scans, kept only as a readable reference. The optimal chain uses an even faster incremental version (section 7).

4. The cheapest block of a given length

4.1 Lemma 2: prefixes

Lemma 2. If a method can encode l frames from p, it can encode any shorter block from p.

Proof. Fixed window: the range of a prefix is not larger than the range of the whole block. Adaptive window: a prefix has fewer frames to fit, and the windows of the frames it has are the same. No compression: always possible. ∎

So method m of channel c can encode the block $p \dots p+l-1$ exactly when $l \le L_{c,m}(p)$: the set of usable methods is "every method that lasts at least l frames". As l grows this set only shrinks.

4.2 The cheapest method of a channel

Among the usable methods of one kind, the narrowest window is the cheapest: for the same l the cost $k + lw$ (or $k + (l-1)w$) only grows with w. By Lemma 1 the usable windows of one kind are exactly the widths from the narrowest usable one upwards. Let

$$w^F(l) = \min\{w : L^F_w(p) \ge l\}, \qquad w^A(l) = \min\{w : L^A_w(p) \ge l\}$$

be the narrowest Fixed and Adaptive windows that last l frames (a kind may have none, then it is left out). The cheapest channel part of the block is

$$\mathrm{cost}^*_c(p,l) = \min\bigl(lk,\ \ k + l\,w^F(l),\ \ k + (l-1)\,w^A(l)\bigr),$$

and the cheapest block of exactly l frames is

$$\mathrm{Bits}(p,l) = H + \sum_{c=0}^{C-1} \mathrm{cost}^*_c(p,l).$$

Why this is exact. Every block that the decoder accepts uses, in every channel, a method that can encode its l frames, so each channel costs at least $\mathrm{cost}^*_c(p,l)$ and the block at least $\mathrm{Bits}(p,l)$. And the block made of the minimizing methods (with the minimum of the block or its first frame as the initial value) is a real block of exactly that size. Note that the channels are independent here: the length l is given, so each channel simply takes its own cheapest usable method.

In the code: `ChannelMethodLengths.GetMinCost()` returns cost^*_c and the method. Its cursors keep w^F and w^A: as l grows they only move to wider windows, so all lengths $1 \dots l_{max}$ cost $O(k)$ cursor moves in total. In case of a tie no compression wins over Fixed, and Fixed over Adaptive; any choice gives the same size.

5. The optimal chain of blocks

5.1 A shortest path

Take the frame boundaries $0, 1, \dots, N$ as the nodes of a graph. A block $p \dots p+l-1$ is an edge $p \to p+l$ with the weight $\mathrm{Bits}(p,l)$, for every $1 \le l \le l_{max}(p)$. Every stream the decoder accepts is a path from 0 to N, and its size is the weight of the path. The edges only go forward, so the shortest path is found by dynamic programming from the end:

$$\mathrm{Best}(N) = 0, \qquad \mathrm{Best}(p) = \min_{1 \le l \le l_{max}(p)} \bigl(\mathrm{Bits}(p,l) + \mathrm{Best}(p+l)\bigr).$$

$\mathrm{Best}(p)$ is computed for $p = N-1, N-2, \dots, 0$, remembering for every p the length l that gave the minimum. Then the chain is read forwards from frame 0: the first block has the length remembered at 0, the next one the length remembered at its first frame, and so on.

Theorem. For the given b, δ and q, the restored chain has the minimum size among all streams that the decoder accepts.

Proof. Any accepted stream is a chain of blocks with $1 \le l_j \le 2^b$ and $\sum_j l_j = N$, and its block at p of length l takes at least $\mathrm{Bits}(p,l)$ (section 4.2). By induction from $p = N$ down to 0: the part of any stream that covers frames $p \dots N-1$ takes at least $\mathrm{Best}(p)$. For $p = N$ it is empty. For $p < N$ it starts with some block of length l, which takes at least $\mathrm{Bits}(p,l)$, and the rest takes at least $\mathrm{Best}(p+l)$; the sum is at least $\mathrm{Best}(p)$ by its definition. The restored chain is made of blocks that take exactly $\mathrm{Bits}(p,l)$ for the remembered lengths, so its size is exactly $\mathrm{Best}(0)$. ∎

The proof needs the blocks to be independent: $\mathrm{Bits}(p,l)$ doesn't depend on what comes before p. That is true because every block carries its own methods and initial values (section 2.2).

5.2 Why the most efficient block is not enough

Versions 1 (sprints 2–3) built the chain from efficient blocks. A block with a high efficiency can still be a bad choice: it may leave a tail that compresses badly, while a short, weak block could open the way to a much better one. The example on the deck (8-bit values `97 97 117 117 117 116 116 117 117 117`, $b = 3$, $H = 7$):

- the most efficient first block is 8 frames with Fixed $w = 5$: $7 + 8 + 8 \cdot 5 = 55$ bits ($E = 14.1\%$), and the last 2 frames then take $7 + 8 = 15$ bits: **70 bits**;
- the optimal chain starts with the weak block `97 97` (Fixed $w = 0$: $7 + 8 = 15$ bits, $E = 6.25\%$), and then all 8 remaining frames fit Fixed $w = 1$: $7 + 8 + 8 = 23$ bits: **38 bits**.

5.3 Ties and restoring the chain

If several lengths give the same total at p, the shortest one is kept (the comparison is strict). Restoring the chain needs the methods of every block, which the dynamic programming doesn't keep: they are found again for the start of each block with a forward scan and `GetMinCost()`. Any method of the minimum cost gives the same size, so the output size is exactly $\mathrm{Best}(0)$.

In the code: `BuildOptimalBlocks()` runs the search and restores the chain; `FindOptimalChain()`/`FindOptimalChains()` are the backward pass; `ChainSearch.Step()` computes $\mathrm{Best}(p)$ for one p. Only the last $2^b + 1$ values of Best are ever needed (no block is longer than 2^b), so they are kept in a ring buffer whose size is a power of two (the index is taken with `& mask`).

6. Computing Bits(p, l) quickly

The minimum over l needs $\mathrm{Bits}(p, l)$ for up to 2^b lengths at every position. Computing each of them from scratch would cost $O(\sum_c k_c)$; instead the lengths are walked in order and Bits is updated.

6.1 Bits(p, l) is piecewise linear

For one channel, at a given l, there are at most three candidates, each a line $\alpha + \sigma l$ (intercept α, slope σ):

candidate	α	σ
no compression	0	k
Fixed, narrowest window $w^F(l)$	k	$w^F(l)$
Adaptive, narrowest window $w^A(l)$	$k - w^A(l)$	$w^A(l)$

The channel uses the lowest of them. While l grows, the lowest line stays the lowest until one of two things happens.

1. Another line becomes cheaper. Only a line i with a *smaller* slope $\sigma_i < \sigma$ can do that: a steeper line falls further behind as l grows. Line i is strictly cheaper when

$$\alpha_i + \sigma_i\, l < \alpha + \sigma\, l \iff l > \frac{\alpha_i - \alpha}{\sigma - \sigma_i},$$

and the first whole l above that fraction is

$$l_\times = \left\lfloor \frac{\alpha_i - \alpha}{\sigma - \sigma_i} \right\rfloor + 1.$$

The "+1" and the strict inequality are there because on a tie the current line is kept. The numerator $\alpha_i - \alpha$ is never negative: the current line is the cheapest at the current l and has the larger slope, so the other line must start higher.

2. A window runs out. The narrowest Fixed window lasts until $l = L^F_{w^F}(p)$; at the next length it can't encode the block any more, the next wider window (with a steeper line) takes its place and the candidates change. The same holds for the Adaptive window. This applies to every candidate, the current line included: if the current line's own window runs out, the segment must end there too.

So the current line of the channel holds up to

$$\mathrm{end}_c = \min\Big(l_\times - 1 \text{ over the flatter lines},\ \ L^F_{w^F}(p),\ \ L^A_{w^A}(p)\Big).$$

Example ($k = 8$). Fixed $w = 3$ lasts until 7 (line $8 + 3l$), Adaptive $w = 4$ lasts until 10 (line $4 + 4l$), no compression is $8l$. At $l = 2$ the Adaptive line is the cheapest (12 against 14 and 16). The Fixed line has the smaller slope, so $l_\times = \lfloor (8-4)/(4-3) \rfloor + 1 = 5$. Indeed, at $l = 4$ the two are equal ($20 = 20$) and at $l = 5$ the Fixed line wins ($23 < 24$). The segment is $l = 2 \ldots \min(4, 7, 10) = 4$.

The segment is not a choice of block length. It is only the range of lengths over which the same line stays the cheapest. The table gives every length; the "cheapest" column is what a comparison of all candidates at every l would give, and the segments give exactly the same costs:

l	**Adaptive** $4 + 4l$	**Fixed** $8 + 3l$	**none** $8l$	**cheapest**	**segment**
2	12	14	16	Adaptive, 12	1 starts: lines compared, $\mathrm{end} = 4$
3	16	17	24	Adaptive, 16	1: $16 = 12 + 4$
4	20	20	32	Adaptive, 20 (tie, current line kept)	1: $20 = 16 + 4$
5	24	23	40	**Fixed, 23**	2 starts: lines compared, $\mathrm{end} = 7$
6	28	26	48	Fixed, 26	2: $26 = 23 + 3$
7	32	29	56	Fixed, 29	2: $29 = 26 + 3$; Fixed $w = 3$ ends

At $l = 5$ the Fixed line is used. The candidates are compared only where a segment starts; inside a segment the next cost is one addition. Which l becomes the block is decided separately, by the dynamic programming of section 5 (the smallest $\mathrm{Bits}(p, l) + \mathrm{Best}(p + l)$).

6.2 The whole block

On a common segment every channel is linear, so the block is linear too:

$$\mathrm{Bits}(p, l) = \alpha_{blk} + \sigma_{blk}\, l, \qquad \alpha_{blk} = H + \sum_c \alpha_c, \quad \sigma_{blk} = \sum_c \sigma_c.$$

The segment of the block ends where the first channel's segment ends, $\min_c \mathrm{end}_c$. Inside it, the next length costs one addition: $\mathrm{Bits}(p, l+1) = \mathrm{Bits}(p, l) + \sigma_{blk}$. At the end of the segment the lines are found again for the new l.

In the code: `ChannelMethodLengths.GetCostLine()` returns α, σ and end of a channel; `ChainSearch.Step()` sums them and walks the segment with `bits += slope`.

7. Pruning: how few lengths are really checked

7.1 Lemma 3: Bits(p, l) never decreases with l

$\mathrm{cost}^*_c(p, l)$ never decreases as l grows: every candidate line has a non-negative slope, and when the candidates change, the set of usable methods only shrinks (Lemma 2), so the minimum can't drop. Hence $\mathrm{Bits}(p, l)$ is non-decreasing in l.

Let T be the best total found so far at p (the smallest $\mathrm{Bits}(p, l) + \mathrm{Best}(p + l)$ over the lengths already checked). Since $\mathrm{Best} \geq 0$:

$$\mathrm{Bits}(p, l) \geq T \;\Rightarrow\; \mathrm{Bits}(p, l') + \mathrm{Best}(p + l') \geq T \quad \text{for all } l' \geq l.$$

So the scan of lengths stops at the first l where the block alone is not smaller than T, and nothing better is lost.

Note what T is: not the cost of a chain built so far, but the best cost found so far for **everything from p to the end of the data** (the scan runs backwards, so at p only the tail $p \ldots N-1$ is known). A single block can reach the cost of the whole tail only when that tail is short compared with 2^b: at the start of the backward scan (the last frames of the file), or in a small file with a large b. In the middle of a large file this stop practically never happens; there the lengths are pruned by the chunk skipping of section 7.2, which also counts the tail after the block.

7.2 Lemma 4: Best(p) never grows with p

Lemma 4. $\mathrm{Best}(p+1) \leq \mathrm{Best}(p)$.

Proof. Take the optimal chain for frames $p \ldots N-1$ and drop frame p from its first block. If that block had one frame, it disappears. Otherwise it becomes a block $p+1 \ldots p+l-1$ with the same methods, which is still valid: a Fixed window's range can only shrink, and an Adaptive block starting one frame later has its new first frame as the initial value and the same pairs of neighboring frames after it. Its cost doesn't grow ($k + (l-1)w \leq k + lw$ and so on). So frames $p+1 \ldots N-1$ can be encoded in at most $\mathrm{Best}(p)$ bits. ∎

Skipping whole ranges of lengths. For the lengths $l \ldots l_2$ of one segment, $\mathrm{Bits}(p, l') \geq \mathrm{Bits}(p, l)$ and, by Lemma 4, $\mathrm{Best}(p + l') \geq \mathrm{Best}(p + l_2)$. So if

$$\mathrm{Bits}(p, l) + \mathrm{Best}(p + l_2) \geq T,$$

none of the lengths $l \ldots l_2$ can beat T and all of them are skipped at once. The code checks chunks of 256 lengths first, then chunks of 8 inside a chunk that can't be skipped, and only then single lengths.

This check fires all the time: once a block is clearly worse than the optimal encoding of the same frames, a whole chunk of 256 (or 8) lengths is rejected with one comparison. The scan still runs up to 2^b, but mostly in chunk steps. For example the text file of the test set with $b = 15$ (32 768 possible lengths) takes about one second.

In the code: `ChainSearch.Step()` (`BigChunk = 256`, `SmallChunk = 8`).

8. Incremental lengths in the backward pass

The dynamic programming visits $p = N-1, N-2, \ldots, 0$ in turn. Instead of scanning the data forward from every p (section 3.3), the lengths at p are updated from those at $p+1$.

8.1 Fixed window

Let $e_w(p) = p + L_w^F(p)$ be the end of the longest Fixed block (the first frame after it). If the block $p \ldots e-1$ can be encoded, so can $p+1 \ldots e-1$ (a smaller range), hence

$$e_w(p) \leq e_w(p+1).$$

So $e_w(p)$ is found by starting from $e = e_w(p+1)$ and moving e to the left while the block $p \ldots e-1$ is longer than 2^b or its range exceeds $2^w - 1$. The minimum and the maximum of the block are kept in two monotonic deques: the new frame x_p is pushed at the front (dropping the front elements that can never be the extremum again), and the frames that leave at the back are removed. Every frame enters and leaves each deque at most once, and e only moves left, so the update costs $O(1)$ per window on average (amortized).

8.2 Adaptive window

An Adaptive block from p breaks at the first frame that doesn't fit the window around its predecessor. Let $\beta_w(p)$ be that frame (or N if there is none):

$$\beta_w(p) = \begin{cases} p+1 & \text{if } x_{p+1} \text{ doesn't fit the window around } x_p, \\ \beta_w(p+1) & \text{otherwise,} \end{cases} \qquad L_w^A(p) = \min\bigl(\beta_w(p) - p,\ l_{max}(p)\bigr).$$

This is one comparison per window and position.

Together: all lengths of all windows for all positions in $O(N \sum_c k_c)$ amortized, instead of rescanning up to 2^b frames for each window at each position.

In the code: `ChannelMethodLengths.CalculateBackward()` with `BackwardState` (`FixedEnd` is e_w, `AdaptiveBreak` is β_w) and `MonotonicDeque`. The deques start small and grow when needed, which keeps the memory independent of 2^b in practice.

9. Several block sizes in one pass

Only $l_{max}(p) = \min(2^b, N - p)$ and the header H depend on b. Because of Lemma 2, the longest block for a smaller b is just the longest block for the largest b_{max}, limited:

$$L_w^{(b)}(p) = \min\bigl(L_w^{(b_{max})}(p),\ 2^b\bigr) \quad \text{for } b \le b_{max}.$$

So one backward pass computes the lengths for b_{max}, and several searches (one per b) share them. Each search uses its own l_{max} and $H = H_{b_{max}} - b_{max} + b$.

In the code: `FindOptimalChains()` (one `ChainSearch` per block size; `BlockSizeBits` is temporarily set to the largest one for the pass).

10. Auto settings

The dynamic programming gives the exact output size, $\mathrm{Best}(0)$, without writing the stream. So the settings can be searched by size:

$$(q^*, \delta^*, b^*) = \arg\min_{q \in Q,\ \delta \in D,\ b \in B} \mathrm{Best}_{q,\delta,b}(0),$$

where Q are the square sides that divide the image width and height (1 for no squares), D the four Δ options and B the offered values of b.

- **All pairs** (q, δ) are tried, in parallel. Squares and the Δ option interact: choosing one and then the other was up to 36% worse on the test set.
- **"Adaptive first" is not computed.** It has exactly the same methods as "Fixed first" (Fixed $0 \dots k-1$, Adaptive $1 \dots k-1$, no compression), only numbered differently, and the same M_c, so every block costs the same. Its size is copied from "Fixed first".
- **The block size** b grows for each pair until the size gets worse 3 times in a row. This is the only heuristic step: in practice the size has a single minimum in b. The next few values of b are computed together (section 9), but never more than the sequential search would compute, so the result is the same.
- **One copy of the data.** All trials read the same frames in the linear order; a trial with squares maps its frame numbers through $\pi(i)$ on the fly. So the memory doesn't grow with the number of trials running in parallel.

The chosen size is the real size of the output, not an estimate.

In the code: `AutoSelectSettings()`, `RunAutoTrials()` (the rounds of b and `SameSizeAs()`), `CreateTrial()`.

11. The frame-by-frame mode (library only)

The application always builds the optimal chain. The library still has the frame-by-frame builder of version 1 (`Compressor.UseOptimalBlockChain = false`), which picks the most efficient block at every position. There a block is a combination of methods $\mathbf{m}$, its length is the shortest of them, $l = \min_c L_{c,m_c}(p)$, and the brute force tried all $\prod_c M_c$

combinations (4096 per frame for RGB888). Version 2 finds the same block exactly, much faster.

Grouping by length. The length of a combination is one of the lengths $L_{c,m}(p)$, so there are at most $\sum_c M_c$ candidate lengths. For a candidate l, let

$$\mathrm{cost}_c^{=}(p,l) = \min_{m:\, L_{c,m}(p)=l} \mathrm{cost}_c(m,l)$$

be the cheapest method of channel c that ends exactly at l (cost_c^{*} is the cheapest that lasts at least l). A combination has length exactly l when every channel lasts at least l and at least one channel j ends exactly at l. For a fixed j the cheapest such combination is $\mathrm{cost}_j^{=} + \sum_{c\neq j} \mathrm{cost}_c^{*}$, and minimizing over j gives

$$\mathrm{Bits}^{=}(p,l) = H + \sum_c \mathrm{cost}_c^{*}(p,l) + \min_j \bigl(\mathrm{cost}_j^{=}(p,l) - \mathrm{cost}_j^{*}(p,l)\bigr).$$

The best block is the candidate with the highest $1 - \mathrm{Bits}^{=}(p,l)/(lF)$. This takes $O((\sum_c M_c)^2)$ instead of $O(\prod_c M_c)$.

Ties. The brute force kept the first combination with the maximum efficiency in the lexicographic order of $\mathbf{m}$. To give the same output byte for byte, the search first finds the maximum efficiency, then, for every length that reaches it, rebuilds the lexicographically first combination of that length and size channel by channel (the smallest method number that still allows the remaining channels to reach the size), and keeps the smallest of these combinations. The second pass recalculates cost^{*} and $\mathrm{cost}^{=}$ for the few lengths that reach the maximum instead of storing them for all candidates, which keeps the first pass free of allocations.

This is not needed by the optimal chain: there the length l is chosen by the search itself, so every channel just takes $\mathrm{cost}_c^{*}(p,l)$, and no channel has to end exactly at l (section 4.2).

In the code: `GetMostEfficientBlock()`, `GetMinBitsForLength()`, `GetFirstCombinationForLength()`, `SmartBuildBlocks()`; all marked *LEGACY* in the code.

12. Complexity

Step	Version 1 (sprint 3)	Version 2
Lengths of all methods at p	$O(\sum_c M_c \cdot 2^b)$	$O(\sum_c k_c)$ amortized
Best block at p (frame-by-frame mode)	$O(\prod_c M_c)$	$O((\sum_c M_c)^2)$
Chain of blocks	greedy, not optimal	exact DP with pruning
Compression time, all test cases	53.7 s	2.9 s

The decoder is unchanged: it reads the same stream format, so all the gains are on the compressor side.

Appendix: where things are in the code

Topic	Section	Methods (`Compressor.cs`)
Frames and squares	2.1	`PrepareFrames`, `GetPixelForSquares`
Header, methods, costs	2.2–2.3	`CalculateHeaderBits`, `GetMethodsCount`, `GetRealMethod`, `GetCompressedBitsForMethod`
Writing a block	2.2	`CompressBlock`
One scan for all widths	3.3	`GetMaxBlockLengths`, `GetMaxBlockLengths3`
Cheapest method	4.2	`ChannelMethodLengths.GetMinCost`
Optimal chain	5	`BuildOptimalBlocks`, `FindOptimalChains`, `ChainSearch.Step`

Topic	Section	Methods (`Compressor.cs`)
Piecewise linear cost	6	`ChannelMethodLengths.GetCostLine`
Pruning	7	`ChainSearch.Step`
Incremental lengths	8	`ChannelMethodLengths.CalculateBackward`, `MonotonicDeque`
Several block sizes	9	`FindOptimalChains`
Auto settings	10	`AutoSelectSettings`, `RunAutoTrials`, `CreateTrial`
Frame-by-frame mode	11	`SmartBuildBlocks`, `GetMostEfficientBlock`, `GetMinBitsForLength`, `GetFirstCombinationForLength`